The Mechanical Eye Ball

Project Report

Project group 1

Sarah Boyd, Edwin Estra, Angel Garcia

8/5/25

Dr. Tamer Omar

Cal Poly Pomona

College of Engineering

ABSTRACT

This project presents the design and implementation of an artificial intelligence tracking system using a Raspberry Pi 5, Raspberry Pi Camera Module v3, Arduino Uno, and two SG90 servo motors. The system uses live video input to detect a desired target and adjusts the camera's pan and tilt angles using servo motors to keep the object centered in the frame. Using the Raspberry Pi's Python Terminal, we are running an ONNX-based machine learning model for object detection, offloaded to a Hailo-8 AI accelerator for optimized performance. Detected positional errors are converted into correction commands and transmitted to the Arduino from the Raspberry Pi, which directly controls the servos within a 0–180° servo range. This project demonstrates the integration of computer vision, embedded systems, and hardware control, highlighting its potential applications in security surveillance, autonomous robotics, and reliability with identifying persons in search and rescue.

TABLE OF CONTENTS

INTRODUCTION

The inspiring motivation behind the Mechanical Eyeball, came from its wide range of uses. One member envisioned a tool for the firefighter department, for search and rescue in dangerous areas. For another member, a tool for security, and so on. The list of uses for a camera to follow and track objects are great in number.

This project focuses on building a low-cost, real-time object tracking system that combines machine learning-based object detection with servo motor control. The goal is to create a platform capable of identifying and following a moving object using a camera mounted on a dual-axis servo mechanism.

The system is built using a Raspberry Pi 5 paired with a Raspberry Pi Camera Module v3 for visual input, and an Arduino Uno to control two SG90 servo motors responsible for panning and tilting the camera. To enable successful object detection, a pre-trained ONNX model is deployed on the Raspberry Pi and accelerated using a Hailo-8 AI module. The Raspberry Pi processes the video stream, determines the object's position relative to the center of the frame, and sends position correction commands to the Arduino via high and low pin communication. The Arduino adjusts the servo angles accordingly to maintain focus on the target, keeping the target within a dead zone where the Raspberry Pi considers the target centered.

This project demonstrates the practical application of embedded AI, mechatronics, and real-time control systems in a cohesive platform. It is a valuable learning experience in combining software and hardware components to solve real-world problems, with potential extensions into fields such as security, car safety, and drones.

EXPERIMENTAL METHODOLOGY

Implementation involves 3 steps:

1. Implement manual control
2. Implement autonomous control
    a. Detect person in the frame
    b. If the person is in the center, do nothing
    c. If the person is on the right side of the frame, turn the camera to the right.
    d. If the person is on the left, turn the camera to the left

3. Tools Used:

- Raspberry Pi 5
- Arduino Uno
- 2x SG90 Servo Motor
- Jumper wires
- Ribbon cables
- ONNX Runtime
- Pre-trained YOLO object detection model
- Pi Camera Module v3
- Python
- Arduino IDE (C++)
- Raspberry Pi Halo AI module
- 4.4v-6v battery
- Breadboard

Please see the list of links to purchase materials, which is located on page 18.

Implementing manual control for the camera was straightforward. We used the Python module “keyboard” to note what key we were holding down. Depending on the key held down (“w”, “a”, “s”, or “d”) the servo motors would move the camera up/left/down/right.

Steps 2 and 3 for autonomous are straightforward. First, we have a servo controlled by an Arduino Uno. The reason why we used the Uno and not the RPi 5 is because the GPIO on the Pi is more of an afterthought, leading to inconsistent performance (in our case, the servo being twitchy). Compare that to the Uno that specializes in being a microcontroller (connecting the servo to the Uno stopped the servo from acting twitchy).

The Pi is still used due to its intuitive interface with its camera and being able to implement computer vision to solve step 1. After the Pi identifies where the person is in the frame, it then communicates to the Uno what direction to turn if need be to keep the person centered.

For our communication between the Pi and the Uno, we made two pins on the GPIO header output; our turn left and turn right signals. These two pins are then connected to another two input pins on the Uno, where the Uno tells the servo to turn left or right, depending which input signal is high. If both input signals are low, the Uno tells the servo to keep its current position.

While steps 2 and 3 were crossed out quickly, step 1 proved to be a major obstacle due to this class being “ECE 3301/L: Intro to Microcontrollers/Lab”, not “ECE 3301/L: Intro to Computer Vision/Lab”. In other words, once we connected the Pi’s camera module (v3) we had a lot of learning to do.

About the Pi’s camera module, we used v3 since one of its features involves being able to automatically focus. We could manually tune the focus if we used a v2, but we decided to use the v3 instead to reduce potential errors that can come up.

Our first goal is being able to identify if a person is in the frame. After doing some research, we use ONNX Runtime to run a pre-trained YOLO model provided by the Ultralytics company because it’s lightweight compared to other options while still being able to run the model quickly. There are many versions of the YOLO models Ultralytics provides, but we used its nano version (yolov5n.onnx) to further keep our model lightweight and ONNX Runtime efficient. Another very important feature ONNX Runtime provides is returning boundary box coordinates, which is what we use to handle what signal to send from the Pi to the Uno.

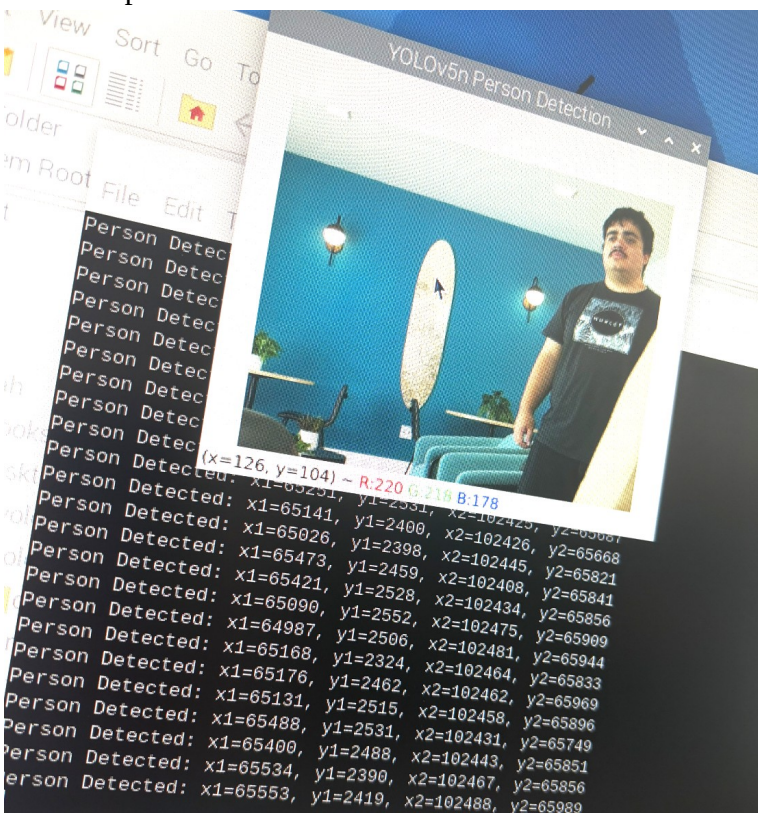
—----------

DESIGN PROCESS

1. Camera and stand

For this project, we used the Raspberry Pi camera V3, with and without the AI module. Pertaining to the visual output, there were two X coordinates and two Y coordinates. The camera receives the input, identifies the moving target that is a person, and creates a boundary box with these coordinates. Using these coordinates, we manually found the horizontal and vertical center of the camera input, and then created a generous range that the target could acceptably be in. This range was used in our code.

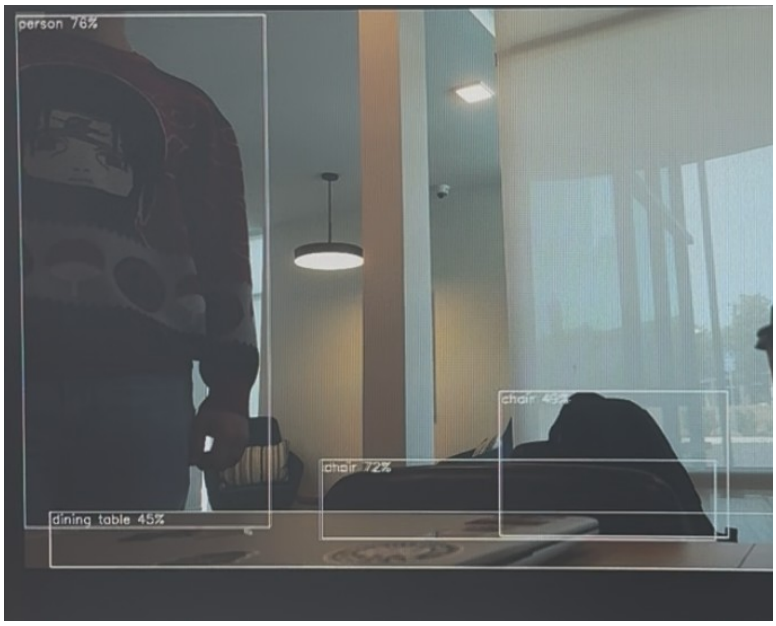
For the horizontal direction, the acceptable range was from 55,000 to 5,000. This means that if x1 > 55,000, the target person is located too far right. As a response, the Pi is coded to put a high on GPIO pin 16, and the arduino receives this high input and is coded to have the pan servo motor move right in response. This is shown in image 1. If x1< 5,000, this means that the person is too far to the left. As a response, the Pi will put a high on GPIO pin 18, and the arduino receives this high input and is coded to move the pan servo motor left in response.



IMG 1. *Example of the output coordinates*

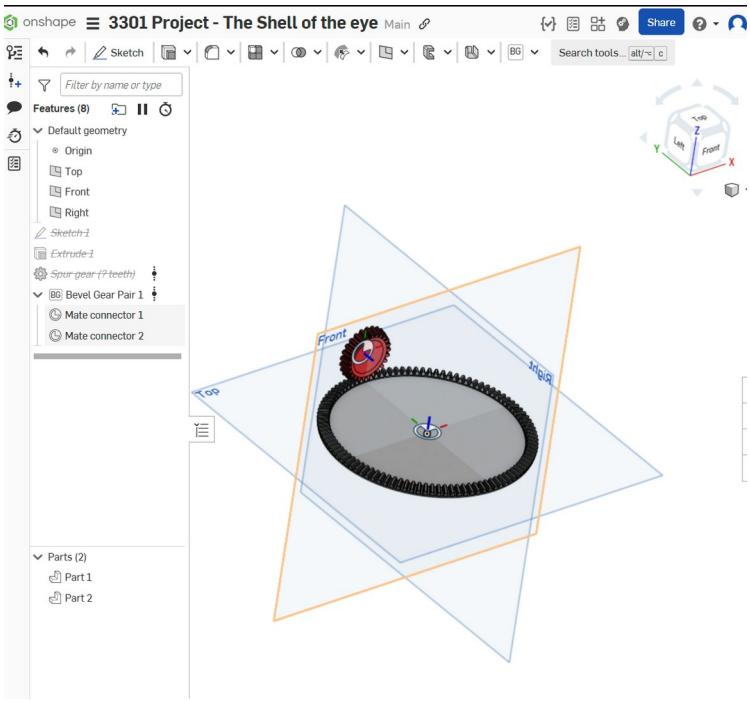
For the vertical range, the acceptable range was from 37,000 to -300. This means that if y1 > 37,000, or the person was down below coordinates with a y value of 37,000, a high would be sent to an output pin going to the arduino and the arduino would know to have the tilt servo motor move down in response. If y1< -300, this would indicate that the person was out of the acceptable range and too high in the visual field, so a high would be sent to an output pin going to the arduino to tell the arduino to move up, in response.

The camera can also connect to the Hailo AI module. This module can use a preloaded library, or a customized library, and identify a wide array of objects. It also smooths camera input and allows for greater, and more efficient, applications.

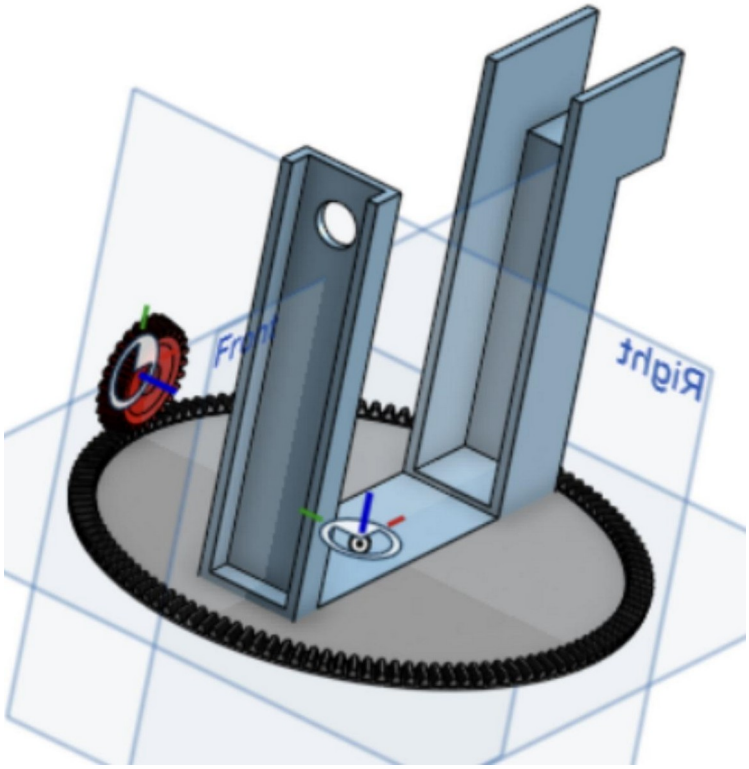


IMG 2. *Identification of objects, using Hailo AI kit.*

A stand that holds the camera and can be adjusted using servo motors was designed. We designed two different versions and found an upgraded third version.



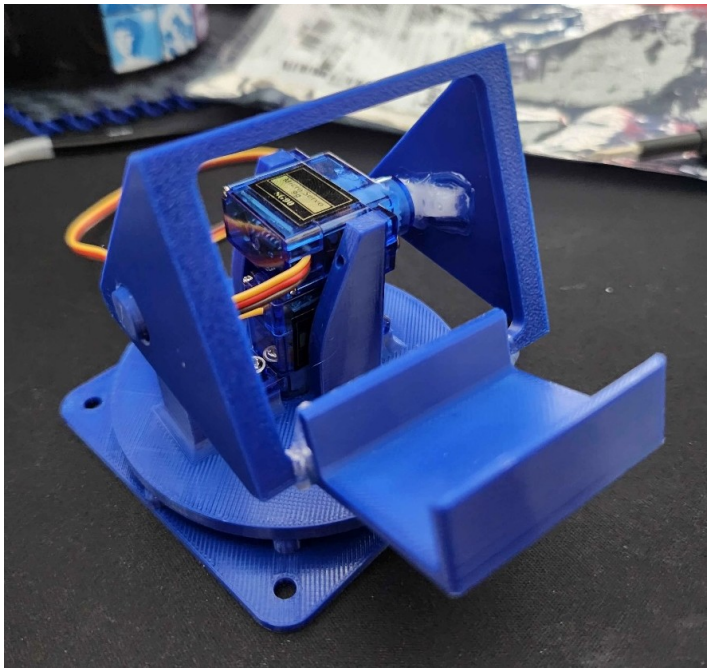
IMG 3. *Designing version one of the camera stand*



IMG 4. *Final product of version one of the camera stand, designed by hand.*

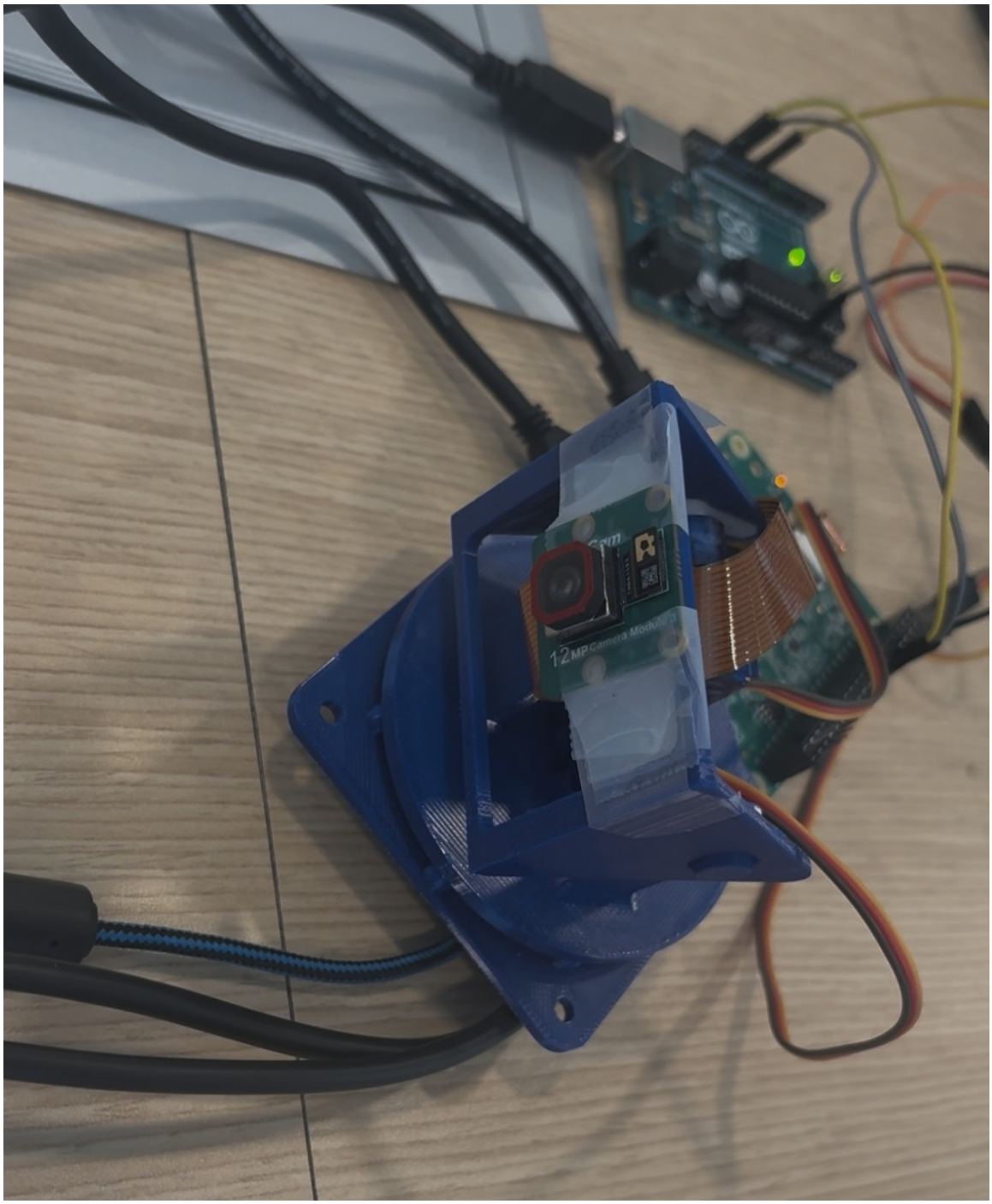
Image 3 shows our design process for designing the first version of the camera stand. It was designed by hand using OnShape 3D CAD software, and was designed with the goal of using a GoPro for visual input. The gears can be manipulated to have more

torque, so version one has interesting potential future uses. The final design for version one of our camera stand can be seen in image 4.



IMG 5. *Version two of the camera stand, modified for our personal needs.*

IMG 6. *Version two of the camera stand with GoPro applications.*



IMG 7. *Version two of the camera stand, as used in our final product.*

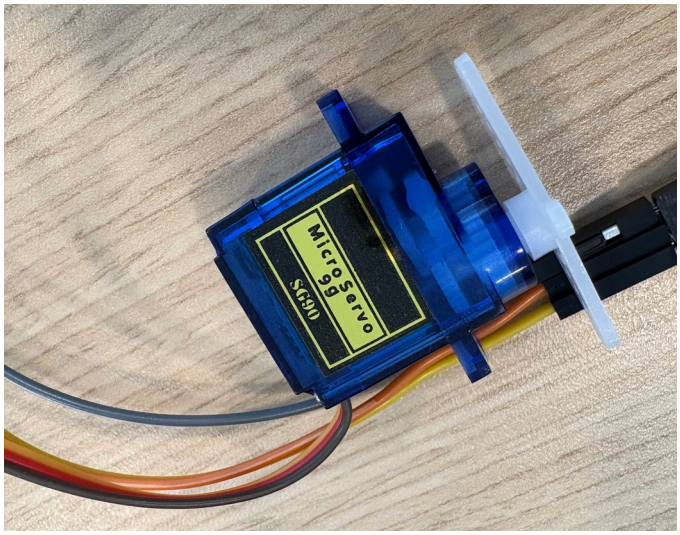
We ended up using a simpler stand, as we decided to use the Raspberry Pi camera V3 instead of the GoPro. Using the RPI camera meant that we would have an easier set up, more software resources and an easier time getting output and coding responses for our input, and lessen the load on the servo motors. Freecad was used to take the model and make it editable on OnShape, as there were some adjustments that needed to be made. The

final usage of the second version of our camera stand can be seen in image 7. The stand used for a GoPro can be seen in image 6. This is included in this report to demonstrate the flexibility of usage of this project and to show that different cameras can be used. The stand we used can be found in the purchase links section, and is the seventh item in our references section.

IMG 8. *Upgraded camera stand with PCA9685 chip. The link to this stand can be found in the Purchase Links section of the report.*

For future applications, a more upgraded camera stand might be desirable. The Raspberry Pi V5 struggles to control servos and camera at the same time, and some stands come with PCA9685 chip to help combat this issue. For our final project, we used an Arduino to separately control the servos based on output for the pi, so this updated stand turned out not to be needed.

2.Servo motors



IMG 9. *SG90 9g motor. Link to product can be found in the purchase links section of the report.*

Two SG90 9g motors, each with three arms, were used- one for tilt (up/down) and one for pan (left/right). Links for these motors are provided in the purchase links section of the report. Since the Raspberry Pi camera V3 is so lightweight, using the SG90 motors is okay despite the cons of these motors. The internal gears are made of plastic and the servos are not hardy, so if we were to use a different camera (such as a GoPro) that necessitates more force in the arms, we could use the MG90 9g servo motors. The MG90 servo motors have metal gears and are more durable, and provide adequate force.

Adjusting the voltage changes the speed, so technically you can control the speed of the motor by putting a large variable resistor in front of it (rheostate) in series to control the voltage. This method is not used with motors because it generates a lot of heat and wastes power. Instead, to control the motors PWM (pulse width modification) is used. Changing the width of the pulse is used to control the average voltage, and therefore control the speed. This works by driving the motor with on/ off pulses and varying the duty cycle (the fraction of time that the output voltage is on compared to when it is off) while maintaining a consistent frequency. Increasing the duty cycle increases the fraction of time that the output voltage is on and increases the average voltage the motor receives. This can be found with the formula:

$$Average\ voltage = (V_{max} - V_{min}) \times Duty\ Cycle\ (\in decimal\ form)$$

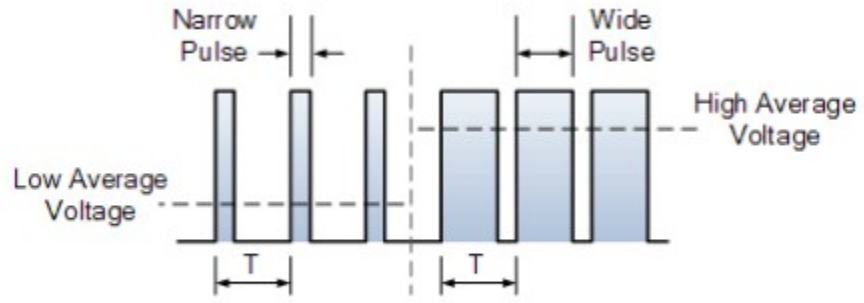
For example, for a maximum voltage of 5v, minimum of 0v, and duty cycle of 50%, $(5v-0v)*0.5=2.5v$ average voltage. This can be seen in image 10.

IMG 10. *Clock wave with duty cycle of 50%. The yellow channel is the output of the scope, and the pink channel is the input.*

Changing the duty cycle to 90% would give us an average voltage of $(5v-0v)*0.9=4.5v$ average voltage. This is also shown in the output of the scope in image 9. Therefore it is shown that increasing the duty cycle from 50% to 90% increases the average voltage.

IMG 11. *Clock wave with duty cycle of 90%*

Therefore, the power applied to the motor can be controlled by varying the width of these applied pulses and therefore varying the average DC voltage applied to the motor terminals. By changing the duty cycle, we control the speed of the motor- the longer the pulse is on, the faster the motor will rotate, and the shorter the pulse is on, the slower the motor will rotate. The wider the pulse width, the more average voltage applied to the motor terminals, the stronger the magnetic flux inside the armature windings is and the faster the motor will rotate.

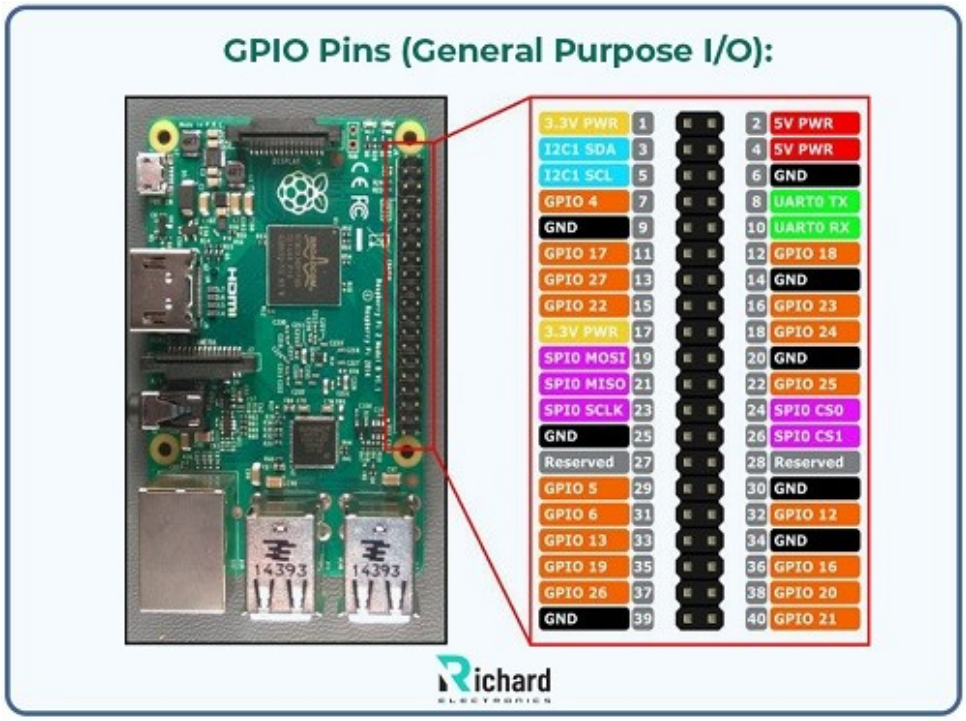


IMG 12. PWM diagram from premierautotrade.com, also found as source 8 in the reference section of the report.

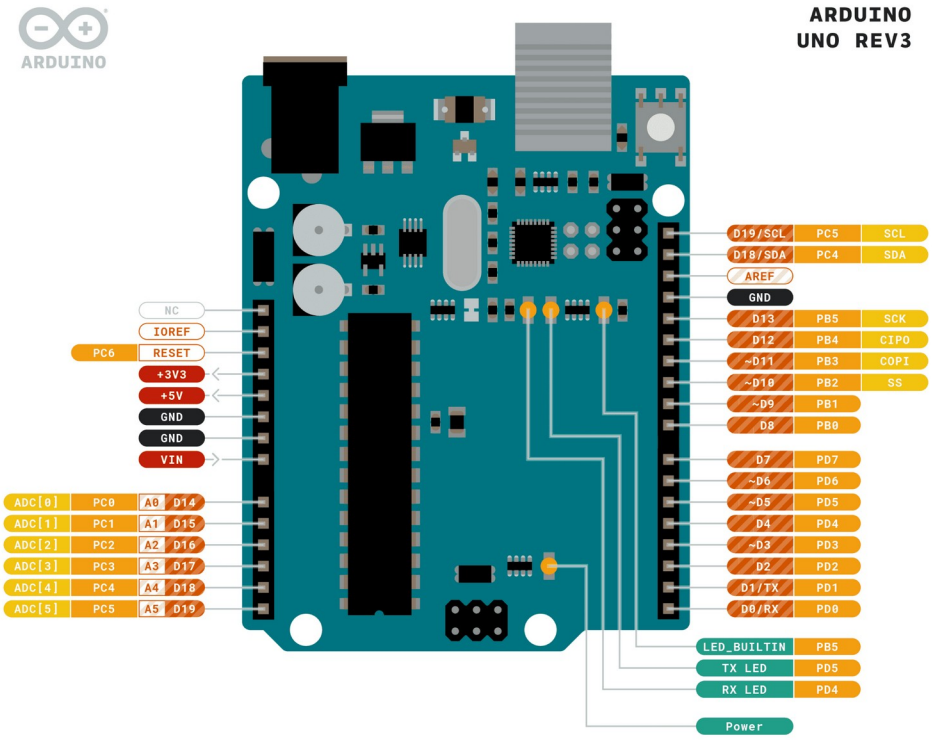
3. Pi and Arduino

For this project we used a Raspberry Pi 5, and connected it to a keyboard, mouse and monitor. It had Raspberry Pi OS pre-installed on a 128GB micro SD card. The link to purchase this product can be found in the purchase links section of this report. The pinout is shown in image 13.

We also used an Arduino Uno. The pinout diagram is shown in image 14



IMG 13. *GPIO pins of the Raspberry Pi 5, found at [richardelectronics.com](). This source is cited in the references section, and is item 9. This source is also an excellent resource for understanding what each pin of the Pi does.*



IMG 14. *Pinout diagram of the Arduino uno, found at [docs.arduino.cc](). The full source is cited in the reference section, and is item 10.*

EXPERIMENTAL RESULTS

We have the camera on a table. When one of us walks across the room, the camera's movements are slightly choppy. While this is due to the camera moving too fast, the camera still tracks where the person goes in the room anyway. As a bonus, we found that if the person runs across the camera, the camera smoothly tracks the person's motion, confirming the reason mentioned above is why the camera movement is choppy when someone is walking across the screen.

The results of this project can be seen in this video: https://youtu.be/xeIDUJbMzKM

SUMMARY OF CHALLENGES AND HOW THEY WERE SOLVED

We began the project by using the Raspberry Pi's AI Hat+ module to use the Hailo accelerator to detect people in the frame. While this was successful, the module did not respond with boundary box coordinates we wanted to use to handle our output signal logic. Due to this problem, we decided to leave the module and use ONNX Runtime instead.

Another problem had to do with communication between the Pi and the Uno. Since we first implemented manual control with the keyboard, the idea was to use UART for the two controllers to communicate with another, where we'd send an "s" or "d" to control the horizontal movement of the camera. However, for reasons unknown, the UART interface did not work because the Uno did not receive any commands sent from the Pi. To solve this problem, we decided to dedicate two GPIO ports to be output for the Pi, which is then connected to two input ports on the Uno. When one of the ports on the Pi is high, one of the input ports on the Uno will also be high, communicating to the Uno to turn the servo left/right.

Finally, when implementing manual control, we found the Pi didn't process keyboard inputs. We found this was due to the library we were using not being compatible with the Pi 5. To fix this, we did some research to find a keyboard input Python module compatible with the Pi 5.

REAL LIFE APPLICATIONS

From a real- world standpoint, our camera- based tracking system offers clear and practical benefits across a large variety of applications. We have created a system that can identify a target and follow a target, even in unpredictable and changing environments. Beyond passive tracking. there is an even greater application with adding responses to the code when a target is identified. For example, the system could use the AI kit library to identify visual input that looks like a fire, the fire can be tracked and maintained at the frame's center, and a mechanism could be triggered in response to put out the fire or alert fire fighters.

Our system is also highly usable in security and surveillance, and can be used anywhere from home security to baby monitors. Our system can also provide visual aid for individuals with impaired vision and has potential to serve as a monitoring system for patients in hospitals. It can be used for managing traffic, analyzing the performance of athletes, and even be used in search and rescue missions. Our project is highly adaptable due to how custom the software can be made, and is useful in an unlimited amount of real-life scenarios. With the low cost, it is possible for people to make their own. Due to its uniqueness, hacking would be difficult, as the information would be foreign and simply coordinates (unless recording was involved).

CONCLUSIONS

While we weren't able to implement vertical control for the autonomous tracking camera, we were still able to track the movement of a person across a room. Future tasks involve tracking vertical movement and implementing dynamic speed so the camera can move smoothly where the person is close to the camera or far away. Also, when it comes to computer vision, the Raspberry Pi 5 isn't amazing because its chip doesn't have hardware acceleration, nor dedicated GPU. At small screen sizes, our app runs just fine, but if we want our app to run on higher resolutions, we have to sacrifice basically every ounce of performance (in terms of FPS). To fix this, using an SBC more specialized in computer vision like the Orin Jetson Nano or an Orange Pi with a Rockchip CPU (since that has hardware acceleration, significantly speeding up video processing) would allow us to use higher resolutions in our app.

PURCHASE LINKS

- Raspberry Pi 5

https://www.amazon.com/dp/B0CTTCVW2D?ref=ppx_yo2ov_dt_b_fed_asin_title

- Jumper wires

https://www.amazon.com/dp/B0BRTJQGS6?ref=ppx_yo2ov_dt_b_fed_asin_title&th=1

- Arduino Uno

https://www.amazon.com/Arduino-A000066-ARDUINO-UNO-R3/dp/B008GRTSV6/?th=1

- 2x SG90 Servo Motor-

https://www.amazon.com/dp/B07Q6JGWNV?ref=ppx_yo2ov_dt_b_fed_asin_title&th=1

- Ribbon cables specifically for the pi-

https://www.amazon.com/dp/B085RW9K13?ref=ppx_yo2ov_dt_b_fed_asin_title&th=1

- Pi Camera Module v3

https://www.amazon.com/InnoMaker-Campatible-Raspberry-Autofocus-Filtter/dp/B0DJ6R67CY

- Raspberry Pi Halo AI module-

https://www.amazon.com/dp/B0D6GMXH73?ref=ppx_yo2ov_dt_b_fed_asin_title

- Camera stand 3D print files free download

Motorized Pan and Tilt Gopro Mount by mopac - Thingiverse

- Upgraded Camera stand with PCA9685 chip-

https://www.amazon.com/dp/B08PK9N9T4?ref=ppx_yo2ov_dt_b_fed_asin_title

This product was not used in our final design, but should be used in future designs where the Pi controls both the servo motors and the camera.

REFERENCES

[1] DIY Engineers. *Master The Raspberry Pi Camera With This Simple Tutorial!*. (Aug. 3, 2024). Accessed: Aug 2, 2025. [Online video]. Available: https://www.youtube.com/watch?v=VeWBYRpodnI
https://www.youtube.com/watch?v=VeWBYRpodnI

[2] Phazer Tech. *Raspberry Pi GPIO Python Guide. (Jan. 17, 2024).* Accessed: Aug 2,2025. [Online video]. Available: https://www.youtube.com/watch?v=n5t8brVw4kQ

[3] Arduino Docs, "Digital Read Serial," docs.arduino.cc. https://docs.arduino.cc/tutorials/uno-rev3/DigitalReadSerial/ (accessed Aug 2, 2025).

[4] Arduino Docs, "Servo Motor Basics," docs.arduino.cc. https://docs.arduino.cc/learn/electronics/servo-motors/ (accessed Aug 2, 2025).

[5] ONNX Runtime, "ONNX Runtime IoT Deployment on Raspberry Pi," onnxruntime.ai. https://onnxruntime.ai/docs/tutorials/iot-edge/rasp-pi-cv.html (accessed Aug 2, 2025).

[6] AntonMaltsev. *Why Raspberry Pi 5 is bad for Computer Vision?*. (Jan 9, 2024). Accessed: Aug. 4, 2025. [Online video]. Available: https://www.youtube.com/shorts/RrWgJlbxhy4

[7] Eman2000. *YouTube*, YouTube, www.youtube.com/watch?v=-iqH7h6Tkxk. Accessed 15 July 2025.

[8]"Premier Auto Trade - Spare Parts, EFI and Engine Management - Mentone, Vic, Australia." *Premier Auto Trade - Spare Parts, EFI and Engine Management - Mentone, VIC, Australia*, premierautotrade.com.au/. Accessed 1 Aug. 2025.

[9]Tianhao, Ling. "Raspberry Pi 5: Pinout, Specs, Datasheet & Projects." *Richard*, Richard Electronics, 19 Nov. 2024, www.richardelectronics.com/blog/projects/raspberry%20pi/raspberry-pi-5-pinout-specs-datasheet-projects.

[10] Arduino. *Docs.Arduino.Cc*, 14 Mar. 2024, docs.arduino.cc/retired/boards/arduino-uno-rev3-with-long-pins/.